An

An Android Application



PocketGuard

<-Your Ultimate Finance Tracking Companion->

Prepared by,

Choyan Mitra Barua Bijoy (Roll: 2007101)
Plaban Das (Roll: 2007111)
Mahir Azmain Haque (Roll: 2007118)

Objectives:

The primary objectives of this project are:

1. Implement secure user registration and login for personalized access.
2. Simplify financial tracking with an easy-to-use transaction panel.
3. Help users set and monitor financial goals effectively.
4. Enable budget creation and management to avoid overspending.
5. Provide parental control features for managing children's finances.
6. Engage users with Firebase push notifications and updates.
7. Offer bookmarking for recommended financial resources.
8. Ensure robust data security and user privacy.
9. Deliver an engaging user experience with an interactive launch.
10. Provide competitive insights fetched from JSON data.
11. Save and fetch transaction invoices from the server.
12. Conduct comprehensive testing for app reliability.
13. Implement scalable and maintainable design patterns.
14. Use ‘Javadoc’ for thorough and clear code documentation.

Introduction to Pocket Guard:

Welcome to Pocket Guard, your easy-to-use finance tracking app. With Pocket Guard, you can easily track your spending, set financial goals, and manage your budget. It also includes features like parental controls to help manage your kids' money, secure logins, and detailed transaction records. Stay updated with push notifications, save useful financial websites, and get insights into competitors. Plus, your data is kept safe and private. Pocket Guard makes managing your money simple and secure, helping you achieve better financial health.

Key functionalities include:

1. Secure User Registration and Login: Allows users to create an account and log in securely, ensuring that personal financial data is protected.

2.Transaction Management: Users can easily add, update, and delete expenses and income, keeping an accurate record of all financial transactions.

3.Goal Setting: Users can set financial goals, such as saving for a vacation or a new gadget, and track their progress towards achieving these goals.

4.Budget Management: Enables users to create and manage budgets for various categories like groceries, entertainment, and transportation, helping them control their spending.

5. Parental Control : Offers features for parents to manage their children's finances, set allowances, and monitor their spending to teach them good financial habits.

6. Recommended Sites and Bookmarks: Provides a list of recommended financial resources and allows users to bookmark their favorite sites for easy access.

7.Firebase Push Notifications :Keeps users informed and engaged with timely alerts and updates about their finances.

8.Foreground Notifications: Ensures that important notifications are delivered even when the app is in use, keeping users aware of critical information.

9.Security Settings: Implements strong security measures to protect user data and maintain privacy, giving users peace of mind.

10.Interactive Launching Activity: Engages users with a dynamic and interactive start-up experience, making the app welcoming and easy to navigate.

11.Competitive Insights: Fetches and displays data about competitors from JSON files, providing users with valuable insights for comparison.

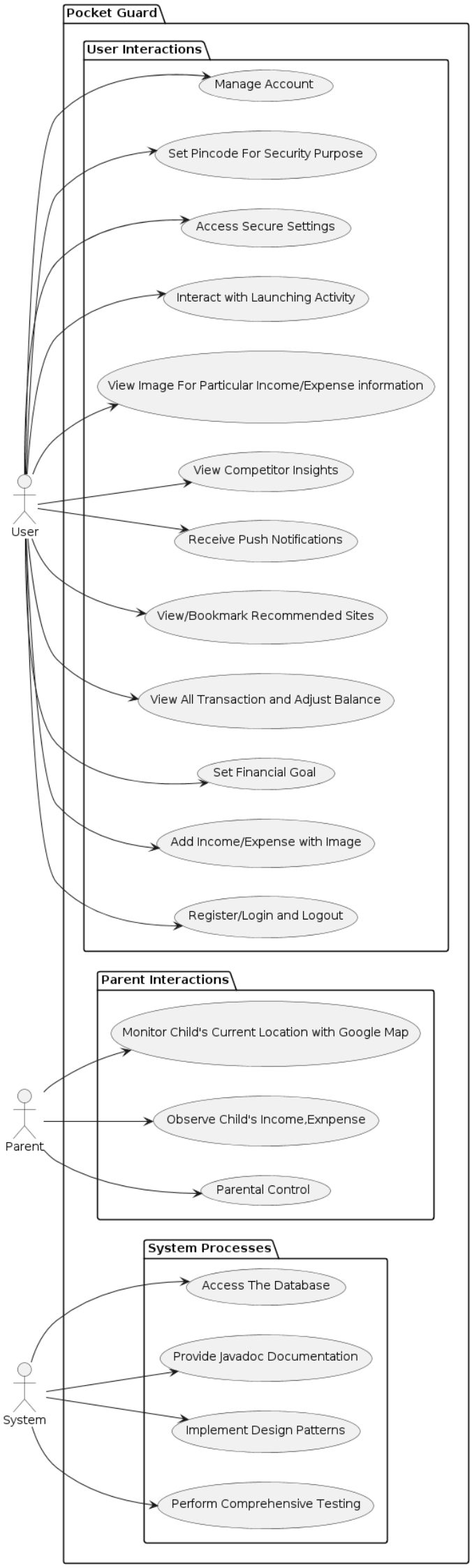
12.Transaction Invoice Management: Saves transaction invoices on the server and allows users to fetch them when needed, ensuring easy access and organization of records.

13.Comprehensive Testing: Ensures the app's reliability and functionality through various types of testing, making sure it works well in different scenarios.

14.Design Patterns: Utilizes different design patterns to make the app scalable and maintainable, ensuring it can grow and adapt over time.

15.Javadoc Documentation: Provides thorough documentation using Javadoc, making the app's code easy to understand and maintain for future development.

Use Case Diagram:



Different Activities:

Login & Registration Section:

The Pocket Guard app's Login and Registration pages handle user authentication and account management functionalities. These pages follow the Model-View-Controller (MVC) design pattern and interact with Firebase for data storage and retrieval.

Login Page

- **Purpose**: Allows users to log into their accounts.
- **Features**:
  - **View Elements**: Email and password input fields, login button, and a link to the registration page.
  - **Controller Logic**: Validates user credentials against Firebase data and starts the `"HomeActivity"` on successful login.
  - **Error Handling**: Provides feedback for incorrect email or password entries.

Registration Page

- **Purpose**: Enables new users to register and existing users to update or delete their accounts.
- **Features**:
  - **View Elements**: Input fields for email, password, name, date of birth, and security question. Buttons for registration, updating, and deleting accounts.
  - **Controller Logic**: Handles user registration by validating input fields and saving user data to Firebase. Supports updating and deleting user information.

- **Fetch Data**: Retrieves and populates user data for updates based on intent extras.

Both pages utilize Firebase Real-time Database for storing and managing user data, ensuring that user authentication and account management are seamlessly integrated with the app's back-end services.

Unit Test : Tested

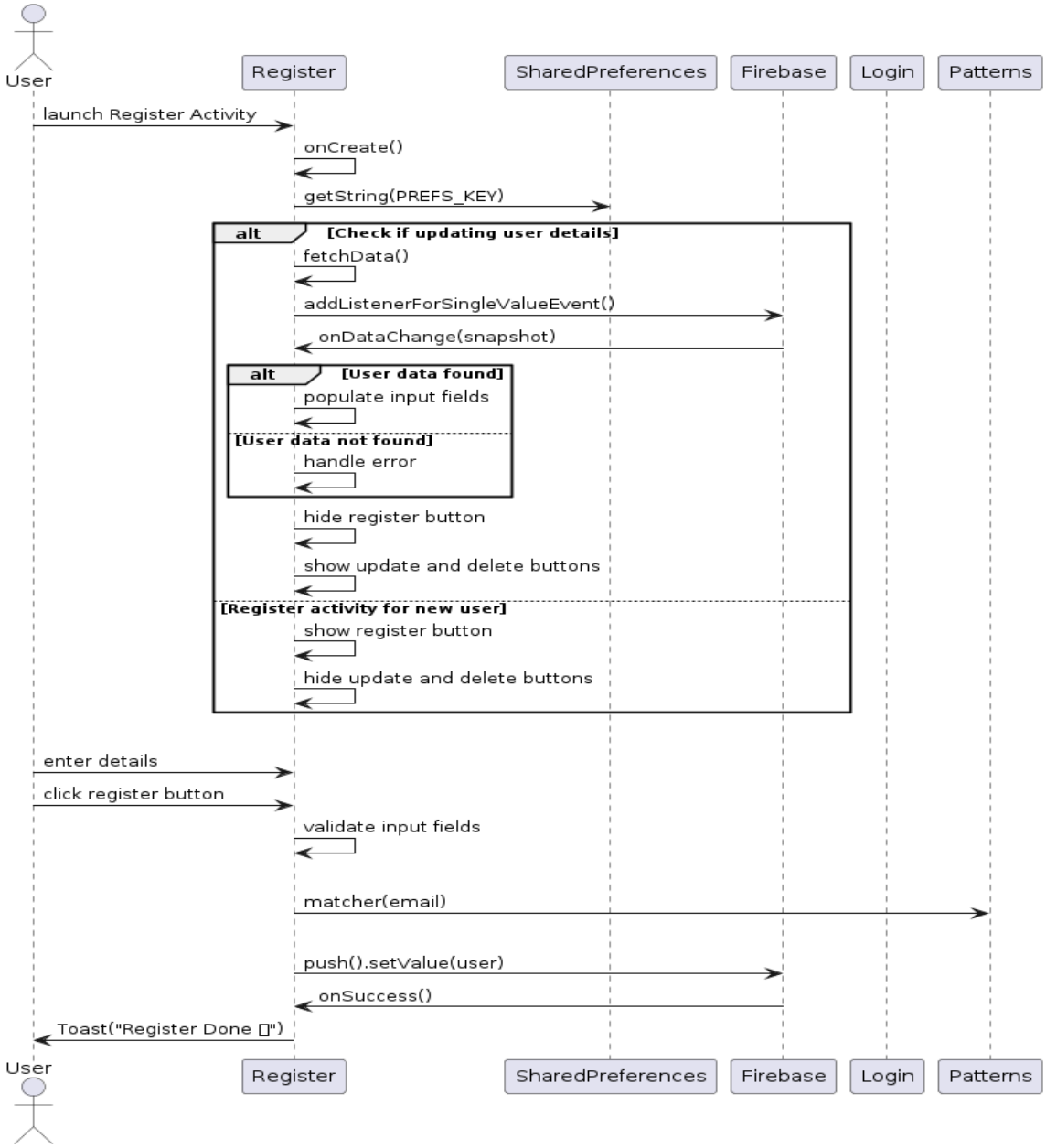
1.testCodeForLogin_SuccessfulLogin
2.testCodeForLogin_FailureLogin
3.testCodeForLogin_EmptyFields
4.testFetchData_Success
5.testFetchData_Failure

UI Test : Tested

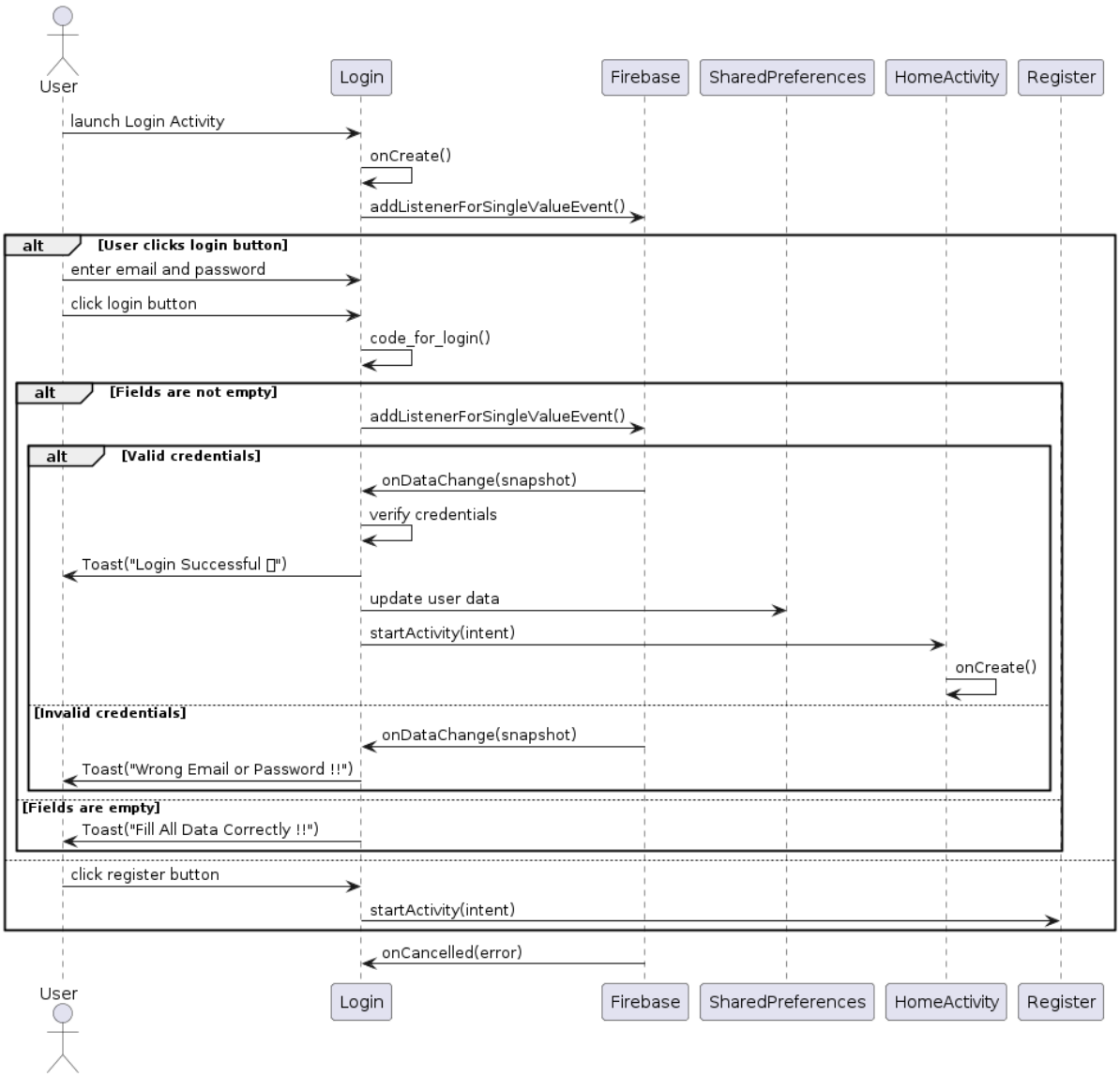
1.testSuccessfulLogin
2.testUnsuccessfulLogin

Javadoc: Generated

Sequence Diagram For Register Activity:



Sequence Diagram For Login Activity:



Bookmarks Section:

The Bookmark Management Activity in Pocket Guard simplifies the way you organize your favorite web pages. It lets you easily save, view, and delete bookmarks. With its user-friendly design, managing your online resources becomes effortless and efficient.

Bookmark Management Activity:

- **Purpose**: Manage bookmarks efficiently.
- **Features**: Store, view, and delete bookmarks.
- **Interaction**: Clickable list items for easy access.
- **Navigation**: Floating action button for quick home screen access.
- **Convenience**: Streamlined user experience with intuitive controls.

Bookmark Page:

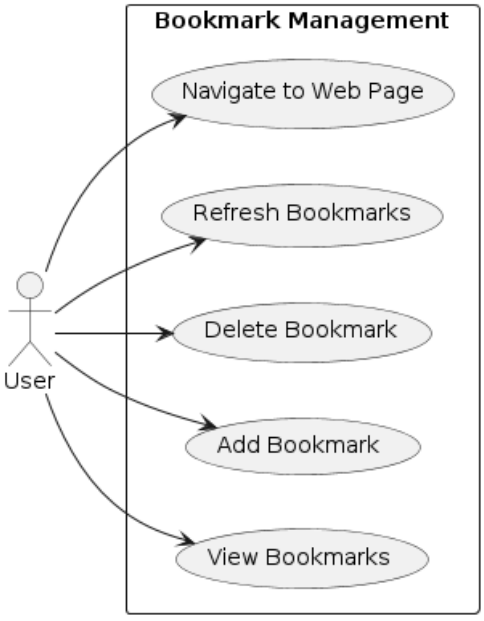
- **Hub for Bookmarks**: Centralized access to saved web links.
- **Functionality**: View, refresh, and delete bookmarks.
- **User-Friendly**: Simple navigation and controls.
- **Efficiency**: Enhances organization of online resources.
- **Convenience**: Seamless integration with home screen navigation.

UI Test : Tested

1.testFloatingButtonVisibility

Javadoc: Generated

UseCase Diagram For Bookmarks:



Goal Management Section:

The “GoalRecyclerView” activity in Pocket Guard helps users handle their financial goals. They can see, create, and remove goals. They can also update the list of goals whenever they want. Plus, users get notifications when they reach halfway to their goal.

This activity uses different design patterns to manage goals efficiently. It organizes code well and makes it easy to add new features.

Functionality:

- Manages financial goals within Pocket Guard.
- Allows viewing, setting, and deleting goals.

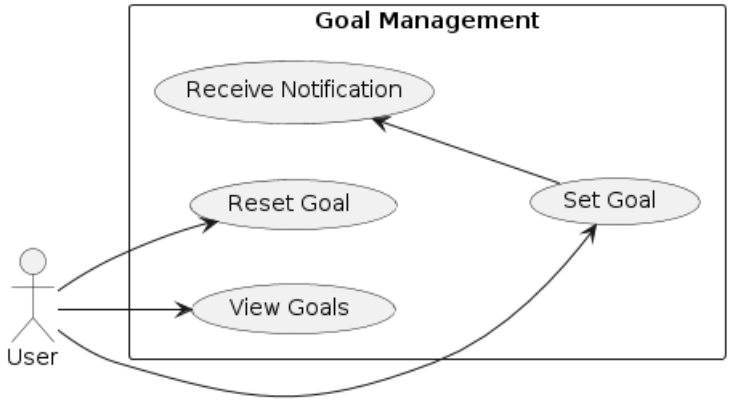
Refresh & Notification:

- Includes refresh option for up-to-date goal list.
- Notifies users upon reaching 50% of their goal.

Integration:

- Interacts with SharedPreferences for persistent data storage.
- Populates RecyclerViews with goal information.

UseCase Diagram For Goal Management:



UI Test : Tested

1.testRecyclerViewItemClick

Javadoc: Generated

Similar Apps Search Activity:

The ”MainActivity_Json” activity in Pocket Guard helps users find and view similar apps along with their ratings and rankings.

Key Features:

- **Fetch Data**: Retrieves a list of similar apps from an API.
- **Display List**: Shows the fetched apps in a ListView.
- **Ratings and Rankings**: Users can click on an app to view its detailed ratings or rank.
- **Search Functionality**: Users can filter the list by entering a search term.
- **Design Patterns**:
  - **Singleton**: Uses a single instance of Volley RequestQueue for network requests.
  - **MVC**: Separates data handling (Model), UI updates (View), and user interactions (Controller).

This activity efficiently displays relevant app information while maintaining clear code structure and organization.

Design Pattern : Added

Javadoc: Generated

Parental Control Activity:

The `Parental_Control` activity in Pocket Guard enables parents to track their child's expenses and current location. This feature ensures that parents can monitor their child's financial and physical whereabouts effectively.

Key Features:

1. **Retrieve Child's Data**:
    - Input required: Child's email and parental control code.
    - Retrieves child's latitude, longitude, total balance, total income, and total expenses from Firebase Real-time Database.
2. **Display Information**:
    - Shows the child's location coordinates (latitude and longitude).
    - Displays the child's total balance, total income, and total expenses.
3. **Google Maps Integration**:
    - Uses Google Maps API to show the child's current location based on the retrieved latitude and longitude.
    - Marker is placed at the child's location with a custom icon.

Design Patterns Used:

1. **Model-View-Controller (MVC)**:
    - **Model**: User data retrieved from Firebase.

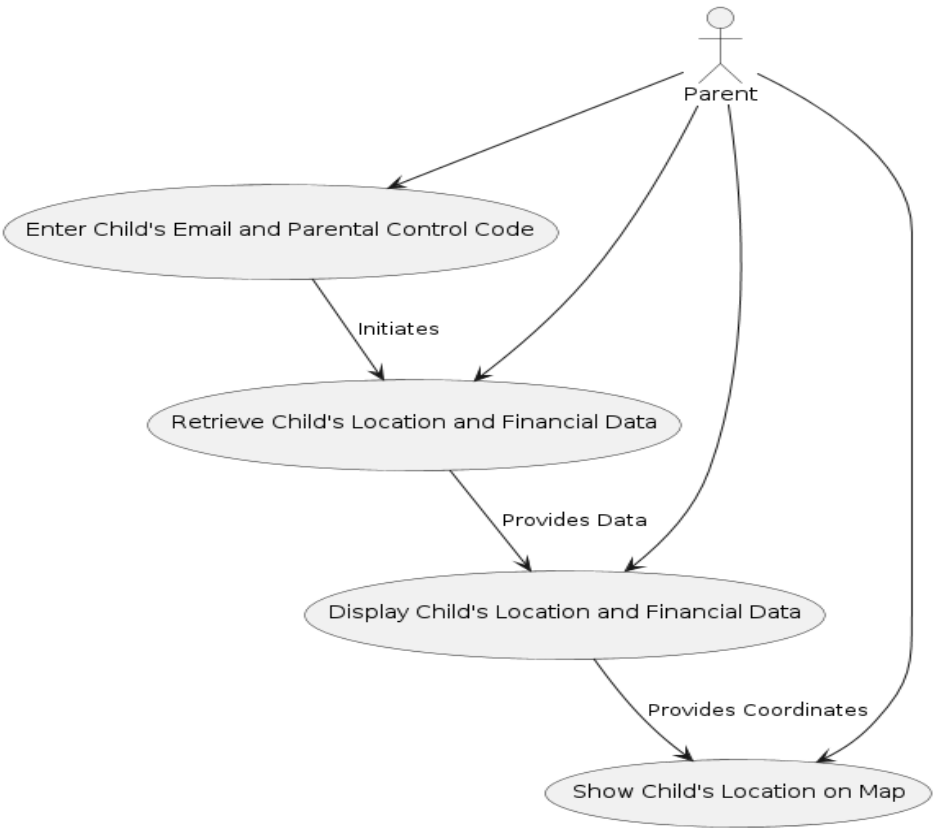
- **View**: UI elements including Text-Views and Google Map.
- **Controller**: `Parental_Control` activity handling user input and data display.

2. **Observer**:

- Uses `”ValueEventListener”` to listen for changes in the Firebase database.
- Implements `”OnMapReadyCallback”` to observe when the Google Map is ready.

This activity ensures parents have real-time access to their child's financial and location data, providing peace of mind and better control over their child's well-being

UseCase Diagram For Parental Control:



Secure Your App with a 4-Digit PIN Code:

Feature Overview:

PocketGuard app includes a security feature that allows users to add a 4-digit PIN code to secure the app. This feature can be enabled or disabled through the app settings. Here's a brief description of how it works:

Enabling PIN Code

1. **Access Settings**: Navigate to the app settings.
2. **Enable Password**: Toggle the "Enable Password" switch.
3. **Set PIN Code**: You will be prompted to enter a 4-digit PIN code and then confirm it by entering it again.
4. **PIN Code Activation**: Once set, the PIN code is stored securely.

Using the PIN Code

1. **App Launch**: Each time you open the PocketGuard app after the splash screen, you will be required to enter the 4-digit PIN code.
2. **PIN Verification**: Enter the correct PIN code to gain access to the app. If the PIN is incorrect, an error message will be displayed, and you will be prompted to try again.

Disabling PIN Code

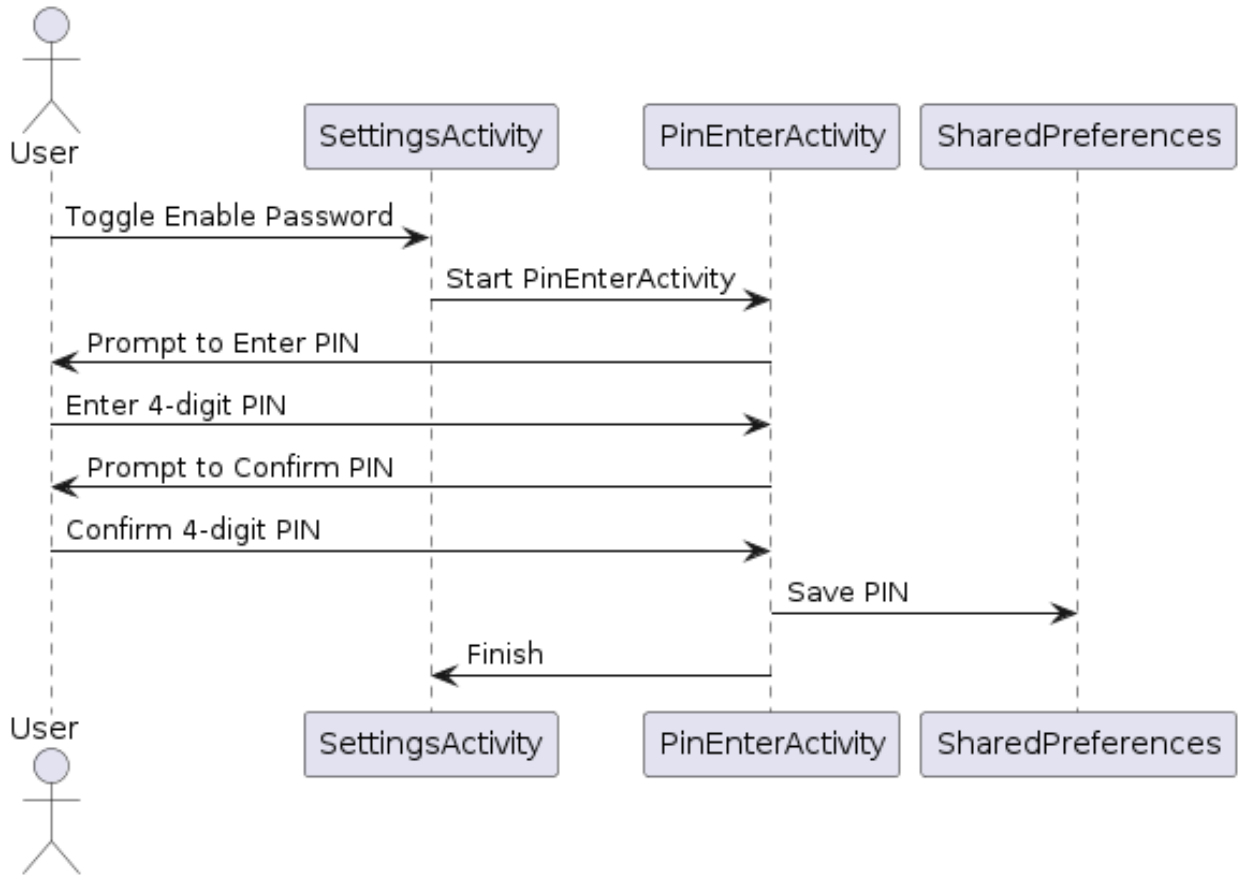
1. **Access Settings**: Navigate to the app settings.
2. **Disable Password**: Toggle the "Enable Password" switch off.
3. **Remove PIN Code**: The stored PIN code will be deleted, and the app will no longer prompt for a PIN on startup.

Factory Reset Option

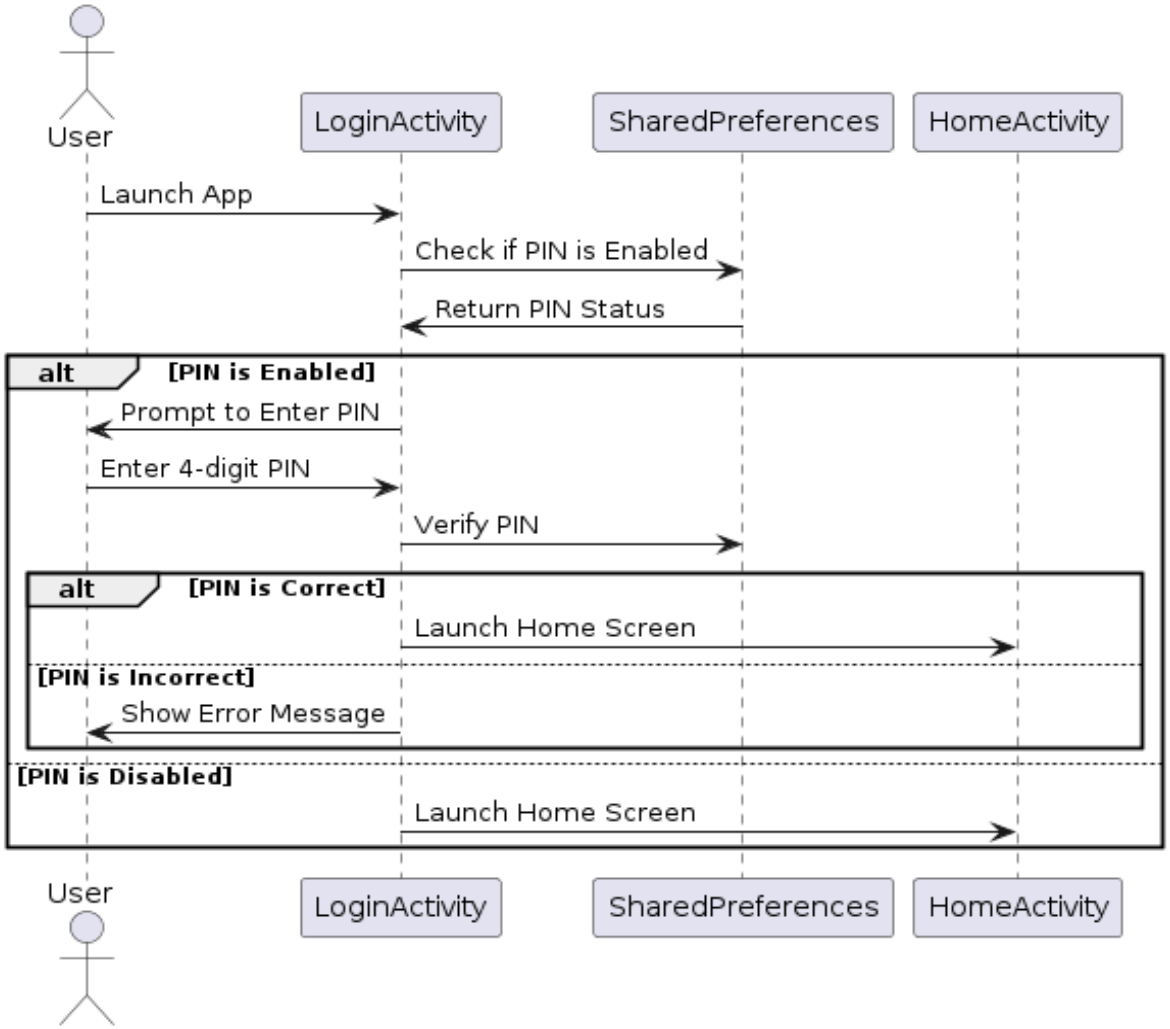
- **Reset PIN Code**: If you forget your PIN, there is an option to reset it by navigating to the factory reset settings. This will remove the existing PIN code, allowing you to set a new one.

This security feature ensures that your PocketGuard app and personal financial information remain protected, giving you peace of mind.

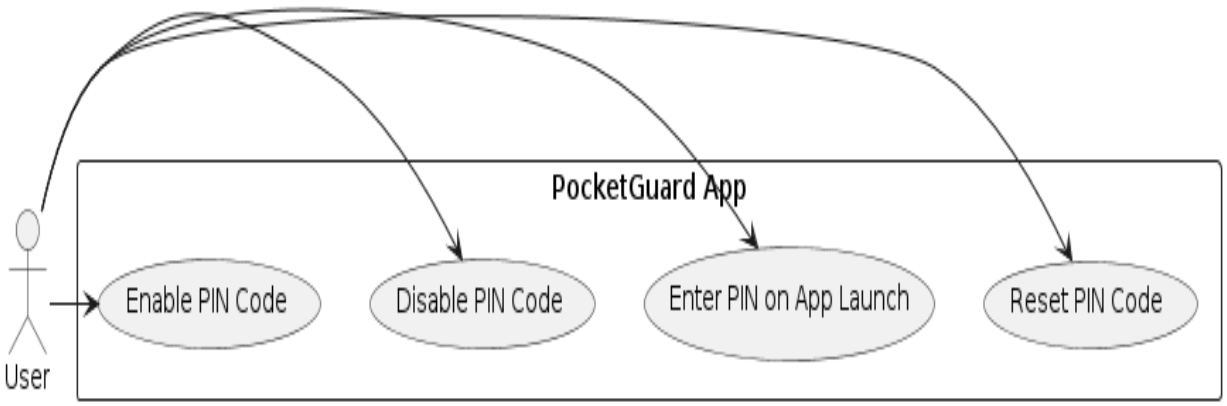
Sequence Diagram For : Enabling PIN Code



Sequence Diagram For : Entering PIN on App Launch



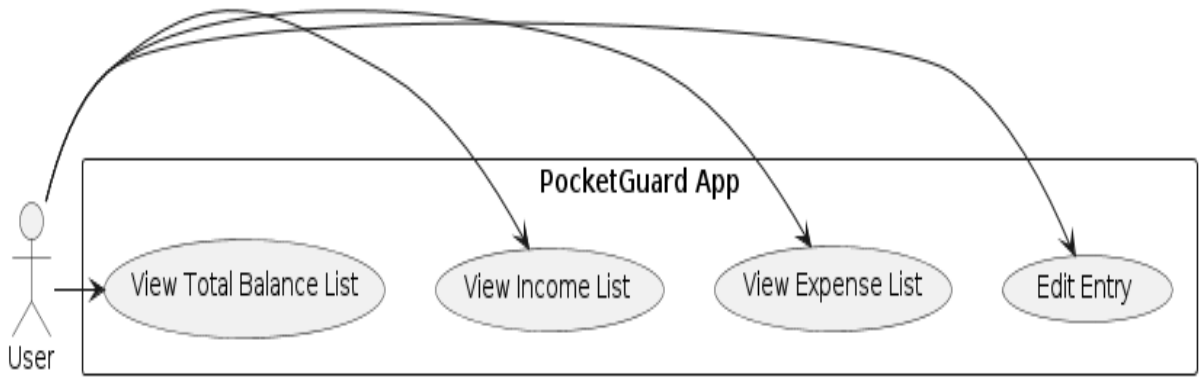
UseCase Diagram for PinCode Activity



ShowData(Total Balance/Expense/Income):

S**qlite_Adapter** and **Sqlite_ListActivity**. The **Sqlite_Adapter** class extends **ArrayAdapter<Sqlite_Model>** and is responsible for adapting SQLite data into a ListView, implementing the Adapter and ViewHolder design patterns for optimal performance, and populating list items with details such as category, method, date, note, and amount, differentiating between income and expense items with distinct icons and text formatting. The **Sqlite_ListActivity** class displays data from an SQLite database in a ListView, using the **Sqlite_Adapter** to bind data, and supports viewing total balance, income, and expense lists. It provides functionalities for editing entries via item clicks, which launch the **AddExpenceIncomeActivity**, and for deleting records through a context menu. The class employs various design patterns, including Observer for handling item clicks, Adapter for data adaptation, Singleton for managing shared preferences, and Builder for dialog creation. The use cases include viewing the total balance list, viewing income and expense lists, editing entries, and deleting entries.

UseCase Diagram for ShowData:



Recommended Websites:

All_Web_Pages_NoAds Activity:

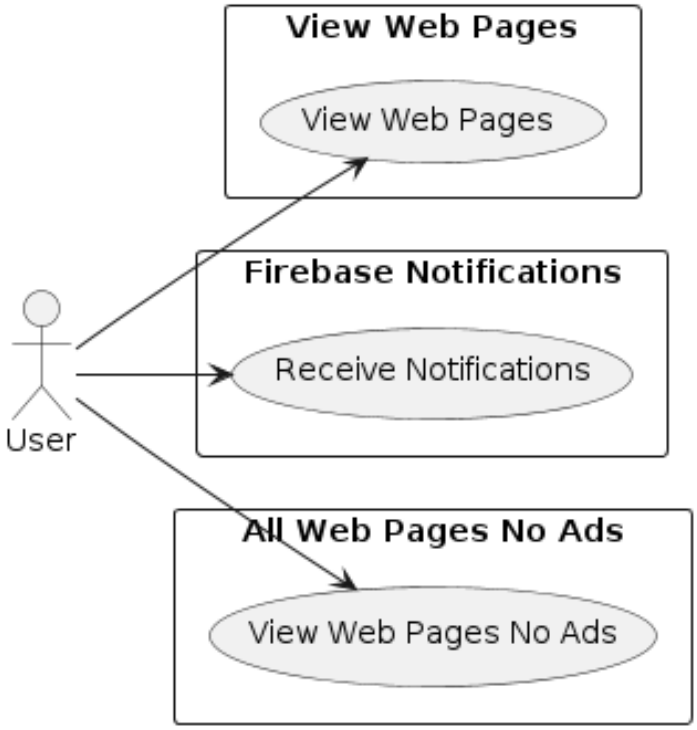
- This activity acts like a browser inside the app, letting you visit websites without annoying ads.
- It sets up a special window (called a WebView) to show web pages and makes sure everything works smoothly, like allowing you to zoom in, enabling interactive features, and showing loading progress.
- You can refresh the page by pulling down the screen, just like you do in some other apps.
- It also lets you bookmark pages you like, so you can easily find them later.
- When you press the back button, it takes you back to the previous page you visited, just like a regular browser.

Firebase_Notification_List_Class Activity:

- This activity shows you a list of notifications that come from a special place called Firebase.
- It keeps an eye on Firebase to see if there are any new notifications and shows them to you when they arrive.
- You can tap on a notification to open the web page it talks about.
- There's a button that takes you back to the main screen of the app when you press it.
- If something goes wrong while it's trying to get notifications, it lets you know politely.

These activities make it easy for you to browse the web and stay updated with notifications, all within the app. They're designed to work smoothly and give you a hassle-free experience.

UseCase Diagram for WebPages:



Adding Income/Expense with Image Based on Goal

This activity allows users to log their financial transactions, both income and expenses, while associating each entry with an image and linking it to a predefined financial goal. Here’s a detailed breakdown of the process:

1. **Transaction Entry**:
    - **Income/Expense Selection**: Users start by choosing whether they are logging an income or an expense.
    - **Amount Entry**: Enter the amount of the transaction.
    - **Category Selection**: Select the category that best describes the transaction (e.g., salary, groceries, entertainment).
    - **Date and Time**: Specify the date and time of the transaction.
2. **Image Attachment**:
    - **Upload Image**: Users can upload an image related to the transaction. This could be a receipt, an invoice, a product image, or any relevant photo.
    - **Image Description**: Optionally, users can add a description to the image for further context.
3. **Goal Association**:
    - **Select Goal**: Users link the transaction to a specific financial goal they have set. For instance, if the goal is to save for a vacation, any savings (income) or spending related to this goal can be associated.
    - **Progress Tracking**: The activity updates the progress towards the goal, showing how the transaction impacts the overall goal (e.g., how much closer the user is to saving the required amount for the vacation).
4. **Review and Confirm**:
    - **Summary Display**: A summary of the entered transaction, attached image, and the linked goal is displayed for the user to review.

- **Confirmation**: Users confirm the entry, and it is saved in their financial records.

5. **Analysis and Reports**:
    - **View Entries**: Users can view a list of all their transactions, filtered by income, expenses, or specific goals.
    - **Progress Reports**: Generate reports to analyze spending habits, income sources, and progress towards goals over time.

This activity not only helps in keeping track of financial transactions but also provides visual evidence and context through images, making the record-keeping more comprehensive and easier to review. Associating transactions with goals enhances financial planning and motivation to achieve set financial targets.

Home Activity Interface: Easy Financial Management :

Main Features:

Manage Transactions:

- **Add Income/Expense**: Quickly enter any money you earn or spend. Choose if it's income or expense, enter the amount, pick a category, and set the date and time. You can also add a picture (like a receipt or invoice) to each entry.
- **View Transactions**: See all your past transactions in one list. You can sort them by type (income or expense), date, category, or linked financial goals.

Add Pictures:

- **Upload Images**: Attach images to your transactions directly from your device. This could be a receipt, an invoice, or anything else related to the transaction.
- **Describe Images**: Add a short description to each image to explain what it is.
-

Set Financial Goals:

- **Create and Manage Goals**: Set specific financial goals, like saving for a vacation, buying a new gadget, or building an emergency fund. Set the target amount and deadline.
- **Link Transactions to Goals**: Connect your income or expenses to these goals to track how each transaction affects your progress.
-

Track Progress and Generate Reports:

- **Monitor Progress**: See how close you are to reaching each financial goal. Track your savings or spending progress visually.
- **Generate Reports**: Create detailed reports to see your spending habits, where your income comes from, and how well you're meeting your goals. Customize these reports by date, category, or specific goals.

Easy Navigation:

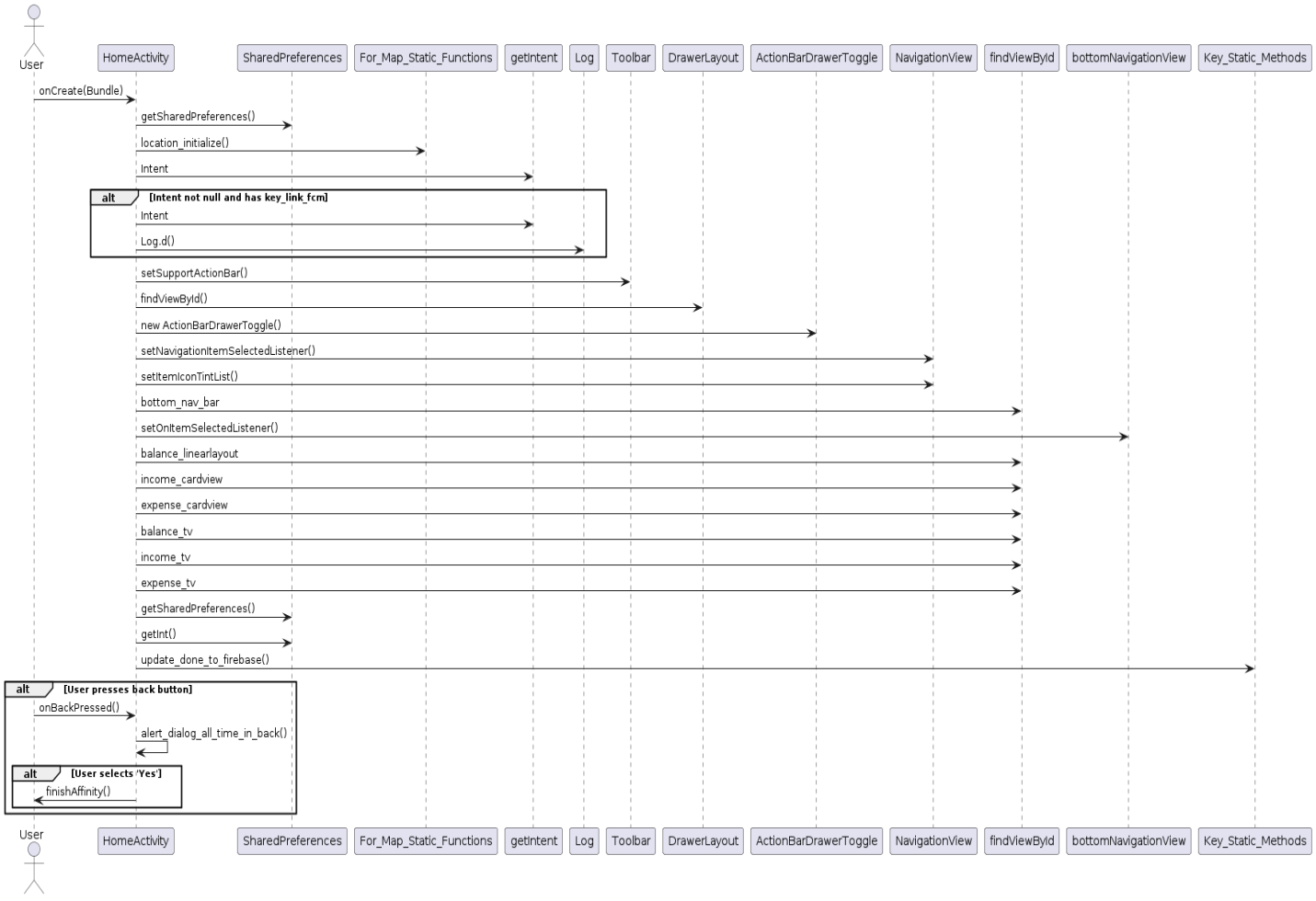
- **User-Friendly Interface**: Move through different features easily. Everything you need is simple to find and use.
- **Quick Links**: Use quick links to jump directly to important tasks, like adding a transaction, viewing history, or updating goals.

Enhanced Experience:

- **Personal Dashboard**: See a summary of your recent transactions, goal progress, and upcoming financial targets on your main dashboard.
- **Notifications**: Get reminders for important things, like upcoming bill payments or goal deadlines.
- **Security**: Your financial information is stored securely and kept private.
-

This home activity interface helps you manage your money by tracking your transactions, setting goals, and monitoring your progress, all in one easy-to-use place. Whether you're keeping track of daily expenses, saving for something special, or just watching your overall financial health, this interface has everything you need.

Sequence Diagram For : HomeActivity



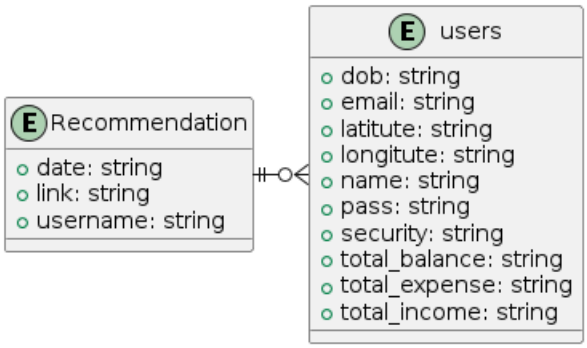
FCM Message Receiver Service:

The “FCMMessageReceiverService” class is like a helper that listens for incoming messages sent through Firebase Cloud Messaging (FCM). When it receives a message, it does a few things:

1. **Receives Message:** It catches messages sent from a server via Firebase.
2. **Processes Message:** It figures out what the message says, like its title, content, and any attached image.
3. **Creates Notification:** Based on the message, it creates a notification that pops up on your phone screen.
4. **Handles Changes:** It also manages changes, like when messages are deleted or when a new token (like an ID) is created by Firebase.
5. **Injects Data:** If there's any extra information in the message, like a link, it makes sure to save it for later, so the app can use it.

Basically, it's the part of the app that makes sure you get notified properly when there's something new or important to see.

Firebase Database:



Design Patterns We Have Used:

Applying Singleton Pattern to Sqlite_StudentDatabaseHelper:

1.Private Constructor:

```
private Sqlite_StudentDatabaseHelper(@Nullable Context context) {
    super(context, DATABASE_NAME, null, DATABASE_VERSION);
}
```

2.Static Instance:

```
private static Sqlite_StudentDatabaseHelper instance;
```

3. Synchronized getInstance Method:

```
public static synchronized Sqlite_StudentDatabaseHelper
getInstance(Context context) {
    if (instance == null) {
        instance = new
Sqlite_StudentDatabaseHelper(context.getApplicationContext());
    }
    return instance;}
```

4. Database Management Methods:

```
@Override
public void onCreate(SQLiteDatabase db) {
    db.execSQL(CREATE_TABLE);
}
```

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int
newVersion) {
   db.execSQL("DROP TABLE IF EXISTS " + STUDENT_TABLE);
   this.onCreate(db);
}
```

Observer Pattern in Your Parental_Control Class:

Code Explanation:
In your code, the Parental_Control activity is set up to observe changes in the Firebase Real-time Database. The ValueEventListener interface methods onDataChange and onCancelled are the mechanisms through which the observer is notified of changes.

```
DatabaseReference db =
FirebaseDatabase.getInstance().getReference("users");
db.addListenerForSingleValueEvent(new ValueEventListener() {
   @Override
   public void onDataChange(@NonNull DataSnapshot snapshot) {
      try {
         // Logic to handle data changes
      } catch (Exception e) {
         e.printStackTrace();
      }
   }

   @Override
   public void onCancelled(@NonNull DatabaseError error) {
      // Handle database error
   }});
```

....And Many More

Testing We Have Done:

UI Test:

These UI tests cover various functionalities and user interactions within different activities of the PocketGuard application:

1. **BookmarksTest**: This test verifies the visibility of the floating action button (`bottom_nav_home_bookmark`) in the `Sqlite_StudentListActivity`. It ensures that the button is displayed as expected.
2. **GoalSetTest**: This test validates the functionality of the RecyclerView item click in the `GoalRecyclerView` activity. It simulates clicking on the first item in the RecyclerView, entering an amount in the dialog, and submitting it. Then, it checks if the entered amount is displayed in the RecyclerView.
3. **HomeActivityTest**: This test focuses on the `HomeActivity` and verifies specific views with different text content. It checks two instances of the `balance_tv` TextView to ensure they display the expected text.
4. **LoginTest**: This test covers the login functionality in the `Login` activity. It includes two test cases: one for successful login and one for unsuccessful login. It enters email and password, clicks the login button, and checks if the appropriate toast message is displayed based on the login result.
5. **Parental_ControlTest**: This test verifies the `Parental_Control` activity's functionality, specifically the "Get Location And Balance" feature. It enters child email and

security code, clicks the "Get Location And Balance" button, waits for certain Text-Views to become visible, and then verifies the displayed location and balance details.

System Development Life Cycle for Personal Finance App

1. Recognition of Need

Objective: Identify the necessity for the app and define its key features.

- Secure user registration and login.
- Transaction panel.
- Financial goal setting.
- Budget management.
- Parental controls.
- Push notifications.
- Bookmarking resources.
- Data security and privacy.
- Interactive user experience.
- Competitive insights.
- Transaction invoice management.
- Comprehensive testing.
- Scalable design patterns.
- Thorough documentation with Javadoc.

2. Feasibility Study

Objective: Assess technical, operational, and financial feasibility.

- **Technical**: Evaluate technology stack (Android SDK, Firebase).
- **Operational**: Assess user needs and workflow integration.

- **Financial**: Estimate development and maintenance costs.

3. Analysis

Objective: Define detailed requirements.

- **User Registration and Login**: Secure authentication.
- **Transaction Panel**: UI/UX design.
- **Financial Goals**: Setting and tracking.
- **Budget Management**: Creation and alerts.
- **Parental Controls**: Account linking and limits.
- **Push Notifications**: Firebase integration.
- **Bookmarks**: Save and access resources.
- **Data Security**: Encryption and privacy.
- **User Experience**: Interactive design.
- **Competitive Insights**: JSON data integration.
- **Transaction Invoices**: Server management.
- **Testing**: Plan for comprehensive testing.
- **Documentation**: Javadoc for clarity.

4. Design

Objective: Create detailed designs.

- **System Design**: Scalable architecture (MVC).
- **Database**: User, transaction, and goal schemas.
- **APIs**: RESTful endpoints.
- **UI/UX Design**: Wireframes and user flow.
- **Security Design**: Data encryption and compliance.

5. Implementation

Objective: Develop the app.

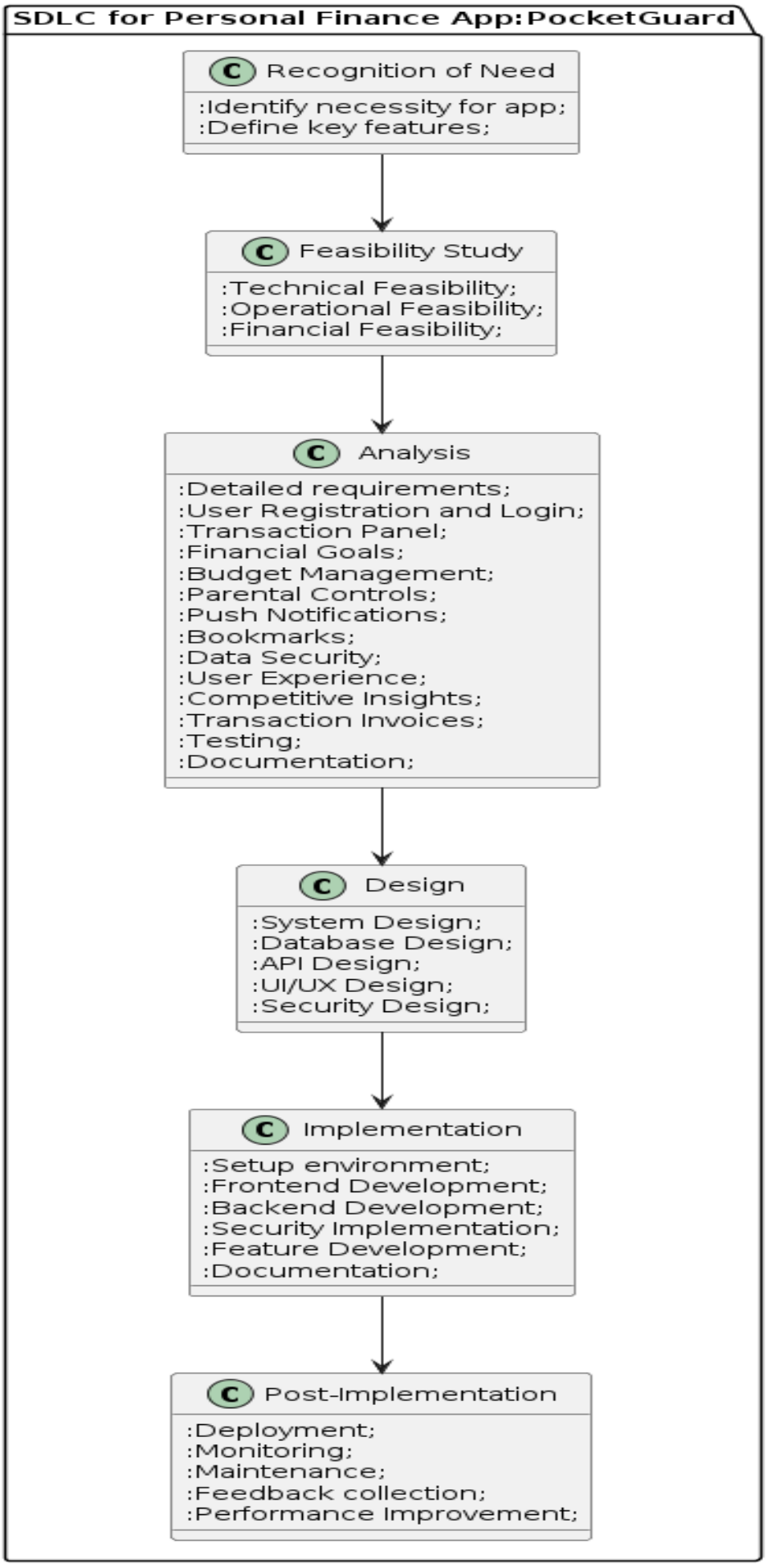
- **Setup**: Configure environment.
- **Frontend**: UI components.
- **Backend**: Firebase and API integration.
- **Security**: Authentication and encryption.
- **Feature Development**: Implement all features.
- **Documentation**: Javadoc for all code.

6. Post-Implementation

Objective: Deploy, monitor, maintain, and update.

- **Deployment**: Publish on Google Play.
- **Monitoring**: Use Firebase Analytics.
- **Maintenance**: User support and updates.
- **Feedback**: Collect and act on user feedback.
- **Performance**: Monitor and improve app performance.

This concise SDLC ensures the personal finance app is systematically developed, high-quality, secure, and user-friendly.


SDLC for Personal Finance App:PocketGuard
Recognition of Need
:Identify necessity for app;
:Define key features;
Feasibility Study
:Technical Feasibility;
:Operational Feasibility;
:Financial Feasibility;
Analysis
:Detailed requirements;
:User Registration and Login;
:Transaction Panel;
:Financial Goals;
:Budget Management;
:Parental Controls;
:Push Notifications;
:Bookmarks;
:Data Security;
:User Experience;
:Competitive Insights;
:Transaction Invoices;
:Testing;
:Documentation;
Design
:System Design;
:Database Design;
:API Design;
:UI/UX Design;
:Security Design;
Implementation
:Setup environment;
:Frontend Development;
:Backend Development;
:Security Implementation;
:Feature Development;
:Documentation;
Post-Implementation
:Deployment;
:Monitoring;
:Maintenance;
:Feedback collection;
:Performance Improvement;

Conclusion

PocketGuard is a personal finance app designed to help users manage their money easily and securely. It includes features like safe user registration and login, easy transaction tracking, budget management, and setting financial goals. It also offers parental controls and sends notifications through Firebase. With a focus on data security and user privacy, PocketGuard aims to make managing finances simple and effective. Comprehensive testing and a scalable design ensure the app is reliable and can grow in the future.